

Spring boot

```
package com.example.demo;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication // annotation
public class DemoApplication {

    public static void main(String[] args) {
        SpringApplication.run(DemoApplication.class, args);

        System.out.println("Application Started");

        // tomcat server
    }
}
```

```
package com.example.demo;

import org.springframework.stereotype.Component;

@Component
public class Student {

    Student(){
        System.out.println("Student class created..");
    }

    int marks = 50;
    void display(){
        System.out.println("Display method from Student class");
    }
}
```

```
package com.example.demo;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
```

```

@RestController
public class Teacher {

    // Field injection

    // @Autowired
    private Student obj ;

    Teacher(Student obj){
        this.obj = obj;
    }

    // setter method

    // public void setter(Student obj){
    //     this.obj = obj;
    // }

    @GetMapping("/home")
    public String greet(){
        // obj.display();
        System.out.println(obj.marks);
        return "HELLO THERE";
    }

    @GetMapping("/data")
    public String data(){
        return "Data method";
    }

}

```

Theory

1. Spring Boot Basics

- **@SpringBootApplication** → Combines @Configuration, @EnableAutoConfiguration, and @ComponentScan.
- **Embedded Tomcat** → Spring Boot runs on an embedded server by default.
- **Main class** → SpringApplication.run() starts the application context.

2. Dependency Injection

- **@Autowired** → Injects beans managed by Spring.
- **Constructor Injection** → Preferred approach (your Teacher(Student obj) constructor).
- **Setter Injection** → Uses setter methods.
- **Field Injection** → Directly injects into fields (less recommended).

3. Components

- **@Component** → Marks a class as a Spring bean.
- Beans are automatically detected by **component scanning**.
- Example: Student class is a bean with marks and display() method.

4. REST Controllers

- **@RestController** → Combines @Controller and @ResponseBody.
- **@GetMapping** → Maps HTTP GET requests to methods.

```
@GetMapping("/home")
public String greet() {
    return "HELLO THERE";
}
```

Coding Round Questions

- Spring Boot basics → What does @SpringBootApplication do?
- Dependency injection → Explain constructor vs field injection.
- Components → What is the role of @Component?
- REST controllers → How do you expose endpoints using @RestController?
- Bean lifecycle → Explain how Spring manages beans.

Machine Coding Round Challenge

Problem: Student API

Requirements:

1. Create a Student class with fields id, name, marks.
2. Annotate it with @Component.
3. Create a StudentController with endpoints:
 - GET /student → return student details.
 - POST /student → accept student data and print it.
4. Use **constructor injection** to inject the Student bean.